Reinforcement Learning-Based Optimal Setpoint Determination in Energy Utility Systems

Master Thesis

Master of Science in Artificial Intelligence

submitted by

Ashish Thapa

5008270

born on 04 March 1999 in Kaski, Nepal

AI-Lab
Chair of Applied Mathematics
Brandenburg University of Technology Cottbus-Senftenberg

Reviewer:	Title and name of first reviewer
Reviewer:	Title and name of second reviewer
Submission date:	19 September 2026

Declaration of independence

I hereby declare that I have written this thesis independently, without unauthorised help and exclusively using the sources indicated. The ideas taken directly or indirectly from external sources (including electronic sources) are labelled as such without exception. The sources used verbatim and in terms of content have been cited in accordance with the recognised rules of scientific work. This work has not yet been submitted for any other review or published elsewhere.

I agree that the thesis will be checked for plagiarism with the help of a plagiarism detection service.

Cottbus, 19 September 2026

Signature of the author

Task description

Energy utility systems are the backbone of industries. Their complexity ranges from small, localized systems to vast, highly interconnected infrastructure networks. Most of them operate under dynamic conditions where multiple interacting variables influence operational efficiency, cost, reliability, and stability. Conventionally, optimal setpoints for such systems are determined using mathematical optimization. These approaches rely on accurate system models and are typically formulated as constrained optimization problems. While effective for well-defined and moderately sized systems, classical optimization becomes computationally expensive as system complexity increases. Furthermore, time-varying dynamics, uncertainty from the integration of renewable sources, and model-plant mismatches makes it challenging to use purely model-based approaches.

This thesis investigates whether reinforcement learning can find optimal operating setpoints for two case studies of different complexity: a battery energy storage system and a non-linear cookie production factory. For each system, the control task is formulated as a Markov decision process (MDP) with explicit state, action, and reward definitions aligned with operational objectives. The RL agent is trained to produce adaptive setpoint policies that maximize battery profit or minimize cookie-factory operating cost while satisfying process constraints and preserving stable operation. The learned policies are then benchmarked against established mathematical optimization methods to determine where RL provides practical benefits and where its limitations remain.

The evaluation of the proposed method will be conducted through systematic simulation studies on two energy system case studies of varying complexity. Performance will be assessed using quantitative metrics including economic performance, constraint violation frequency, and computational time. The results will provide a structured comparison between reinforcement learning and classical optimization and will determine under which conditions RL offers practical advantages for real-time setpoint optimization in complex energy utility systems.

Abstract

Example summary in English.

Zusammenfassung

Beispielzusammenfassung in Deutsch.

Contents

List of Figures xii

List of Tables xiii

List of Abbreviations xiv

Nomenclature xv

1 Introduction 1

2 Theory and Background 3
2.1 Setpoint Optimization in Energy Utility Systems 3
2.2 Mathematical Optimization 3
2.3 Reinforcement Learning 5
2.3.1 TD3 and SAC 7

3 Use Cases 9
3.1 Battery Energy Storage System 9
3.2 Cookie Production Factory 11

4 Methodology 13
4.1 Problem Formulation 13
4.2 Reinforcement Learning Framework 13
4.3 Training Setup 15
4.4 Evaluation Metrics 16

5 Results and Discussion 19
5.1 Battery Energy Storage System 19
5.2 Cookie Production Factory 21
5.3 Key Findings 22

6 Conclusion and Future Work 23
6.1 Conclusion 23
6.2 Future Work 24

List of Figures

1 Interaction loop between the reinforcement-learning agent and the environment in a Markov decision process [17]. 5
2 Schematic of the battery energy storage system. The battery charges from the grid and sells electricity back to the grid; its state of charge SOC(t) records the stored energy. 9
3 Schematic representation of the electrified steam-generation system serving the cookie production factory. Reproduced from Kyriakidis et al. [9]; the system was originally introduced by Walden et al. [3]. . 11
4 TD3 and mathematical-optimization objective values for ten randomly selected 24-hour comparison windows. The lower panel shows the corresponding TD3–mathematical-optimization gaps. 20
5 Illustrative 24-hour battery dispatch comparison between the TD3 policy and mathematical optimization. 21
6 Illustrative 24-hour cookie-factory dispatch comparison between the SAC policy and mathematical optimization. “Math” denotes mathematical optimization. Positive net thermal flow indicates TES charging, and negative net thermal flow indicates discharging. 22

List of Tables

2 Main SAC and TD3 training settings used for the two reinforcement-learning environments. TD3 uses the Stable-Baselines3 default network architecture; the remaining listed settings are shared. 16
3 Comparison between the trained SAC and TD3 policies and mathematical optimization over 1000 random 24-hour windows. Mathematical optimization serves as the baseline; lower (more negative) objective values correspond to higher profit. 19
4 Comparison of the trained SAC policy and mathematical optimization over ten matched 24-hour cookie-factory windows. Lower values are better for the cost and constraint-violation metrics. 21

List of Abbreviations

Abbreviation	Definition
AI	Artificial Intelligence
ANN	Artificial Neural Network
BESS	Battery Energy Storage System
BO	Bayesian Optimization
CPU	Central Processing Unit
EUR	Euro
HTHP	High-Temperature Heat Pump
IPOPT	Interior Point OPTimizer
KKT	Karush–Kuhn–Tucker
MDP	Markov Decision Process
MLP	Multilayer Perceptron
NLP	Nonlinear Programming
RL	Reinforcement Learning
SAC	Soft Actor-Critic
SG	Steam Generator
SOC	State of Charge
TD3	Twin Delayed Deep Deterministic Policy Gradient
TES	Thermal Energy Storage
WT	Wind Turbine

Nomenclature

Symbols are grouped by context. A dash denotes a dimensionless quantity; "varies" denotes a quantity whose units depend on the model, scaling, or constraint. Time is expressed in hours for electricity-cost and battery-energy calculations and in seconds for the thermal-storage balance when heat rates are in kW and heat capacities in kJ/(kg K).

Symbol	Definition	Unit
Time, energy, and economic quantities		
t, i	Discrete time index and summation index	–
N	Number of decision steps in the scheduling horizon	–
n	Number of time steps in the cookie-factory formulation	–
Δt	Duration of one time step	h or s
S_t	Spot electricity price	EUR/MWh
P_t^{S}	Grid-side electrical power; positive for purchase	kW
W_t	Available wind power	kW
C_t	Grid-electricity cost at one time step	EUR
J, J_{grid}	Total operating or net grid-electricity cost, with the appropriate energy-unit conversion	EUR
Battery energy storage		
$E_{\max}$	Usable battery energy capacity	kWh
$P_{\max}$	Maximum charging and discharging power	kW
$P_{\mathrm{actual},t}$	Effective storage-side power after efficiency losses	kW
u_t	Normalized battery control command	–
SOC_t	Battery state of charge at time step t	–
$\mathrm{SOC}_{\mathrm{final}}$, SOC_N	Final state of charge	–
$\mathrm{SOC}_{\mathrm{term}}$	Required minimum terminal state of charge; also written as $\mathrm{SOC}_{\mathrm{term}}^{\mathrm{min}}$ or $\mathrm{SOC}_{\mathrm{min}}^{\mathrm{terminal}}$	–
$\mathrm{SOC}_t^{\mathrm{reach}}$	Minimum state of charge that preserves terminal-target reachability	–
η	Charging and discharging efficiency	–
k	Steepness parameter of the smooth action and SOC gates	–
δ	Normalized action deadband	–
ϵ	Power scale for the smooth efficiency transition	kW
Cookie production factory and thermal storage		
$T_{\mathrm{I}}^t, T_{\mathrm{III}}^t$	HTHP inlet temperatures at the indicated locations	°C
$T_{\mathrm{II}}^t, T_{\mathrm{IV}}^t$	Heat-exchanger outlet temperatures predicted by the surrogate models	°C
$T_1^t, T_2^t, T_3^t, T_4^t$	Thermal-fluid temperatures at the numbered circuit locations	°C
T_s^t	Thermal energy storage temperature	°C

Symbol	Definition	Unit
$\tilde{T}_0$	Prescribed initial thermal-storage temperature	°C
$\dot{m}_{\mathrm{I}}^t, \dot{m}_{\mathrm{II}}^t$	Heat-transfer-fluid mass flow rates	kg/s
m_s	Mass of the thermal-storage medium	kg
$c_{p,\mathrm{f}}, c_{p,\mathrm{s}}$	Specific heat capacities of the heat-transfer fluid and storage medium	kJ/(kg K)
R^t	Compressor-speed-related control variable	–
β_1^t, β_2^t	Bypass mixing fractions	–
$\dot{Q}_{s,\mathrm{ch}}^t$	Thermal-storage charging heat rate	kW
$\dot{Q}_{s,\mathrm{dch}}^t$	Thermal-storage discharging heat rate	kW
$\epsilon_{\mathrm{ch}}, \epsilon_{\mathrm{dch}}$	Thermal-storage charging and discharging effectiveness	–
γ	Upper bound on the product of charging and discharging heat rates	kW^2
F_{HTHP}	Surrogate model for electrical power demand of one HTHP unit	kW
$F_{\mathrm{HTHX}}, F_{\mathrm{LTHX}}$	Surrogate models for high- and low-temperature heat-exchanger outlet temperatures	°C
Mathematical optimization		
$x, \mathbf{x}, x^\star$	Decision vector and optimal decision vector	varies
Ω	Domain of the decision variables	varies
n, p, q	Numbers of decision variables, equality constraints, and inequality constraints in the general optimization problem	–
$f(x)$	Objective function	varies
$h(x), g(x)$	Equality- and inequality-constraint functions	varies
s, s_j	Slack vector and its jth component	varies
μ	Logarithmic barrier parameter	varies
y, z	Equality- and inequality-constraint multipliers	varies
J_h, J_g	Jacobian matrices of the constraint functions	varies
$S = \mathrm{diag}(s)$	Diagonal matrix of slack variables	varies
Reinforcement learning and evaluation		
$\mathcal{X}, \mathcal{A}$	State and action spaces	varies
x_t, x'	Current state and next state	varies
X_t, A_t	Random state and action at time step t	varies
a_t, a_t^{phys}	Policy action and corresponding physical setpoints	varies
$\mathcal{M}$	Map from policy actions to physical setpoints	varies
$p(x' \mid x, a)$	Transition probability or probability density	varies
$\pi, \pi_\theta(a \mid x)$	Policy and parameterized stochastic policy	varies
$\mu_\theta(x)$	Deterministic TD3 actor	varies
θ, ϕ_1	Actor parameters and first-critic parameters	varies
$\mathcal{D}$	Replay buffer	–
τ_t	Stored transition tuple	varies
d_t	Episode-end indicator	–
$r(x, a), r_t$	Reward function and time-step reward	varies

Symbol	Definition	Unit
r_t^{reach}	Reward including the terminal-SOC reachability penalty	varies
G_t	Discounted return from time step t	varies
γ_{RL}	Reward discount factor	–
v_t^π, q_t^π	State-value and action-value functions under policy π	varies
$v_t^\star, q_t^\star$	Optimal state-value and action-value functions	varies
q_{ϕ_1}	First parameterized critic	varies
$J_{\text{TD3}}, J_{\text{SAC}}$	TD3 actor and SAC policy objectives	varies
α	Weight of the SAC entropy term	varies
$\mathcal{H}$	Policy entropy	–
τ	Target-network smoothing coefficient	–
$v_{\ell,t}, v_{\text{T},t}$	Constraint-violation and terminal-violation terms	varies
$\lambda_\ell, \lambda_{\text{T}}$	Constraint and terminal penalty weights	varies
$\lambda_{\text{SOC}}, \lambda_{\text{reach}}$	Terminal-SOC and reachability penalty weights	varies
R_{ep}	Cumulative undiscounted episode reward	varies
V_{tot}	Sum of recorded constraint-violation magnitudes	varies
f_{viol}	Fraction of time steps with constraint violations	–
ε	Numerical tolerance used in violation detection	varies
$\mathbb{E}, \mathbb{E}_\pi$	Expectation and expectation under policy π	varies
$\mathbb{I}, \mathbf{1}_{\{\cdot\}}$	Indicator function	–

1 Introduction

Modern energy utility systems supply the electricity and heat needed to sustain industrial production and other essential activities. Their efficient operation is important for meeting energy demand while limiting operating costs and environmental impacts. Key objectives of industrial energy management include reducing operating costs and CO_2 emissions, improving energy efficiency, and integrating renewable energy sources. Achieving these objectives requires coordinated decisions about energy conversion, storage, and supply while satisfying the requirements of the processes being served.

These operating decisions are shaped by dynamic conditions and multiple sources of uncertainty. Electricity prices vary over time, renewable generation is weather dependent, and storage systems couple current decisions with future operating flexibility [1]. At the same time, industrial energy systems often contain nonlinear component behavior, operating and coupling constraints that must be respected at every time step. The resulting setpoint selection problem is therefore not only an economic problem, but also a constrained sequential decision-making problem.

In many engineering applications, such setpoints are determined using mathematical optimization. This approach is attractive because physical constraints and objective functions can be formulated explicitly, and mature solvers can compute high-quality solutions for well-defined models. For energy systems, nonlinear programming methods such as IPOPT [2] are commonly used when component models, storage dynamics, and operational constraints are nonlinear [3]. However, deterministic global optimization can become computationally demanding for large-scale or nonconvex problems, making it difficult to obtain a globally optimal solution within a practical time limit [4]. Stochastic global optimization methods can also require many objective-function evaluations and exhibit slow convergence, with computational cost increasing as the problem dimension grows [5]. In industrial energy management, forecast uncertainty, optimization-horizon length, and computation time per rolling-horizon iteration are additional factors that influence system performance [6].

Reinforcement learning (RL) provides a different perspective. Unlike classical control and optimization methods that solve a trajectory-optimization problem from scratch at each operating horizon, RL agents learn a policy or action-value function gradually through interactive experience. Once trained, this learned representation enables rapid selection of actions from the current observation [7]. This makes RL attractive for real-time or repeated decision-making problems, especially when the system is complex, nonlinear, and operates in a changing environment [8]. Nevertheless, RL does not automatically guarantee feasibility or optimality. For engineering use, a learned policy must be evaluated not only by its reward, but also by its operating cost, constraint satisfaction, and computation time relative to established optimization methods.

This thesis investigates whether RL can be used for optimal setpoint determination in energy utility systems of different complexity. Two case studies are considered to examine how the number of control variables, nonlinear component behavior, and coupling between operational constraints affect the applicability of RL. The first is a battery energy storage system (BESS), which represents a comparatively simple storage-control problem with a single charging or discharging command, state-of-charge dynamics, and time-varying electricity prices. The second is the electrified steam-generation utility system of a cookie production factory, incorporating wind

power and thermal energy storage. Based on the benchmark systems described by Walden et al. and Kyriakidis et al. [3, 9], this case has greater complexity because it combines multiple continuous control variables, nonlinear heat-pump and heat-transfer behavior, and coupled mass-flow, temperature, and storage constraints. Comparing the two cases allows the benefits and limitations of RL to be assessed across a simpler storage system and a more complex industrial energy utility system.

The central research question of this thesis is:

> *Can reinforcement learning determine feasible and cost-effective operating setpoints for energy utility systems of varying complexity?*

This question is addressed through the following subquestions:

- How can the setpoint optimization problem be formulated as a Markov decision process with suitable states, actions, and rewards?
- Can deep reinforcement-learning policies learn continuous setpoint decisions that respect the system dynamics and operational constraints of the two systems?
- How does the learned policy compare with mathematical optimization in terms of operating cost, constraint satisfaction, and computation time?
- Under which conditions does RL offer a practical advantage, and where do classical optimization methods remain superior?

For each case study, the setpoint optimization problem is first formulated as a Markov decision process (MDP). Deep neural networks are used to parameterize the RL policies that map system observations to continuous setpoint decisions. The resulting policies are evaluated through deterministic rollouts in reinforcement-learning environments, and the resulting schedules are compared with mathematical optimization baselines. The comparison is based on total grid-electricity cost, constraint violations, and computation time. This separation is important because a policy with high cumulative reward is only useful if it remains feasible and economically meaningful under the original engineering objective.

The remainder of this thesis is structured as follows: Section 2 establishes the theoretical foundations of setpoint optimization and reinforcement learning, and introduces mathematical optimization as the benchmark used to evaluate the RL policies. It also describes the deep RL methods considered in this work. Section 3 introduces the two case studies: a battery energy storage system and a nonlinear cookie production factory. Section 4 presents the methodological framework, covering the MDP formulation, environment design, policy-training procedure, optimization baselines, and evaluation metrics. Section 5 reports and discusses the experimental results, with particular emphasis on the comparison between RL policies and mathematical optimization. Finally, Section 6 summarizes the main findings, identifies the limitations of the study, and outlines directions for future work.

2 Theory and Background

This section provides the theoretical basis for the setpoint optimization problem studied in this thesis. The central question is how operating setpoints in energy utility systems can be selected over time so that costs are reduced while physical and operational limits are respected. The same problem can be viewed from two perspectives: as a model-based constrained optimization problem, and as a sequential decision-making problem that can be addressed with reinforcement learning. The section first introduces the fundamental concepts of RL and the MDP formulation [7], then presents deep RL methods for continuous control and the TD3 [10] and SAC [11] algorithms evaluated in this work.

Here, RL is not treated as a substitute for physical constraints. Instead, the setpoint-optimization problem is formulated as an MDP, with feasibility enforced where possible through action bounds, state-dependent mappings, and system dynamics, and otherwise discouraged through reward penalties. This distinction is important when comparing RL with mathematical optimization, because a low-cost policy is only useful if its decisions remain feasible.

2.1 Setpoint Optimization in Energy Utility Systems

Energy utility systems are operated by choosing setpoints that determine how components such as storage units, heat pumps, valves, and bypasses are controlled. A setpoint describes the desired value of a control variable. In the systems considered here, this may be the charging or discharging power of a battery, the bypass variable β_1, or a mass flow rate. These choices are not just technical details; they directly affect operating cost, energy efficiency, constraint satisfaction, and overall system reliability and lifetime [12].

Setpoint determination is also inherently sequential. The best operating point at one time step cannot be chosen independently of what happened before or what may be needed later. Storage systems make this especially clear. Charging a battery or a thermal energy storage can increase future flexibility, but it may also increase the current electricity cost. Discharging can reduce current grid consumption or generate sale revenue, but it leaves less stored energy for later. Good setpoint decisions must therefore account for inter-temporal dependencies and physical operating limits [3]. Each action affects both the immediate economic outcome and the feasible decisions available at later time steps; consequently, its quality must be evaluated over the full planning horizon rather than solely at the current time step [13].

In practical operation, the controller also has to react to external signals such as electricity prices, renewable generation, production demand, and ambient conditions. Since these signals change over time, fixed rule-based strategies can become inefficient, and the operation can be formulated as a finite-horizon decision problem with time-varying boundary conditions [8, 14]. This motivates two formulations of the same setpoint determination task: constrained finite-horizon optimization and reinforcement learning.

2.2 Mathematical Optimization

Mathematical optimization provides a natural way to describe setpoint selection. In general, optimization means selecting the best decision from a set of feasible alternatives according to a defined objective. For energy systems, the objective is often to minimize operating costs, energy consumption, or CO_2 emissions, while the

feasible set is determined by physical equations and operational limits [3, 6]. A constrained optimization problem can be written as [15]:

$$\min_{x \in \Omega} f(x) \quad \text{s.t.} \quad h(x) = 0,\ g(x) \leq 0, \tag{1}$$

Here, $x \in \Omega \subseteq \mathbb{R}^n$ denotes the decision variables, $f : \mathbb{R}^n \to \mathbb{R}$ is the objective function, $h : \mathbb{R}^n \to \mathbb{R}^p$ represents equality constraints, and $g : \mathbb{R}^n \to \mathbb{R}^q$ represents inequality constraints. The solution $x^\star$ is optimal if it minimizes the objective while satisfying all constraints.

For finite-horizon setpoint optimization, the decision vector contains the control actions for several time steps. The objective is usually the accumulated operating cost over the horizon. The constraints describe system dynamics, energy balances, storage bounds, and terminal requirements. This time-coupled structure is important because a state such as battery state of charge or thermal storage temperature depends on previous decisions.

The difficulty of an optimization problem depends strongly on its mathematical structure. Linear and mixed-integer linear formulations are widely used in energy-system applications because simplified models can often be solved reliably. However, realistic energy conversion systems frequently include nonlinear component behavior, continuous control variables, and coupled dynamic constraints. Such systems lead to nonlinear programming (NLP) formulations, which are generally harder to solve and may contain local optima [15]. As the number of time steps, decision variables, nonlinear component models, and coupling constraints increases, the optimization problem generally becomes larger and more difficult to solve within a short operational time window [9].

Another important distinction is whether the problem is deterministic or stochastic. In deterministic optimization, all relevant parameters, such as electricity prices, wind availability, and demand, are assumed to be known over the optimization horizon. Stochastic optimization represents uncertain quantities through random variables or scenarios. This is closer to real operation, but it also increases model size and computational complexity.

Nonlinear constrained problems are usually solved with dedicated numerical solvers. Interior-point methods are common in this setting. Wächter and Biegler introduce IPOPT as an open-source solver for large-scale nonlinear programming that uses a primal-dual interior-point method with a filter line-search strategy [2].

More advanced hybrid optimization methods, such as Bayesian Optimization-Interior Point OPTimizer (BO-IPOPT), have been developed for complex nonlinear, nonconvex, and constrained optimization problems. BO–IPOPT combines the global exploration capability of Bayesian optimization with the efficient local convergence and constraint handling of IPOPT. The method has been evaluated for the optimal operation of an industrial energy conversion system and subsequently enhanced for constrained problems of varying dimensionality, multimodality, and nonlinearity [9, 16]. In this thesis, IPOPT is used as the mathematical-optimization baseline because of its wide applicability to nonlinear optimization problems. The evaluation of BO-IPOPT as an additional global optimization baseline is left for future work.

To illustrate the interior-point principle, consider the general problem in Eq. (1). Introducing slack variables converts the inequality constraints into $g(x) + s = 0$, where $s \geq 0$. Here, $s = (s_1, \ldots, s_q)^\top$, and s_j is the slack variable associated with the jth inequality $g_j(x) \leq 0$. A logarithmic barrier with parameter $\mu > 0$ restricts

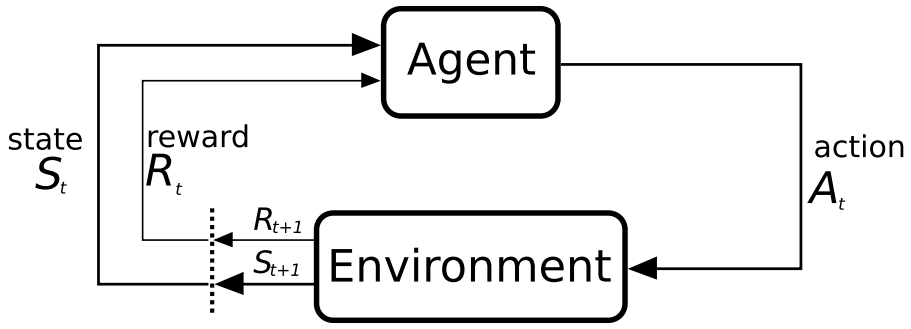


Figure 1: Interaction loop between the reinforcement-learning agent and the environment in a Markov decision process [17].

the iterates to the interior, $s > 0$, and leads to the subproblem

$$\begin{aligned} \min_{x,s} \quad & f(x) - \mu \sum_{j=1}^{q} \ln(s_j) \\ \text{s.t.} \quad & h(x) = 0, \qquad g(x) + s = 0, \qquad s > 0. \end{aligned} \tag{2}$$

The logarithmic barrier prevents the iterates from reaching the constraint boundary. IPOPT solves a sequence of such subproblems using a primal-dual Newton method. With equality multipliers y and inequality multipliers z, the perturbed Karush-Kuhn-Tucker(KKT) conditions are

$$\begin{aligned} \nabla f(x) + J_h(x)^\top y + J_g(x)^\top z = 0, & \\ h(x) = 0, \qquad g(x) + s = 0, & \\ Sz = \mu \mathbf{1}, \qquad s > 0,\ z > 0, & \end{aligned} \tag{3}$$

where J_h and J_g are the Jacobians of the equality and inequality constraints, respectively, and $S = \operatorname{diag}(s)$. As μ is reduced towards zero, the perturbed complementarity condition $Sz = \mu \mathbf{1}$ approaches the KKT complementarity condition of the original constrained problem [2].

Nocedal and Wright note that the performance of nonlinear optimization algorithms depends on formulation quality, particularly the constraint structure, variable scaling, and initial point [15].

These considerations motivate the investigation of reinforcement learning as a complementary approach to model-based optimization. The following section introduces its fundamental principles and its formulation for setpoint optimization.

2.3 Reinforcement Learning

Reinforcement learning (RL) offers a different view of the same setpoint problem. Instead of solving an optimization problem from scratch at every decision time, an agent learns from repeated interaction with an environment. At each time step, the agent observes the current state, selects an action, receives a reward, and observes the next state. Over time, it adapts its policy so that actions leading to high long-term reward become more likely [7].

Formally, RL problems are commonly modeled as a Markov Decision Process (MDP), defined by the tuple [7]

$$(\mathcal{X}, \mathcal{A}, p, r, \gamma_{\mathrm{RL}}),$$

where

- $\mathcal{X}$ is the set of states representing the system,
- $\mathcal{A}$ is the set of actions,
- $p(x' \mid x, a)$ defines the transition dynamics,
- $r(x, a)$ is the reward function,
- $\gamma_{\mathrm{RL}} \in [0, 1]$ is the discount factor.

Under the Markov property, the current state contains all information required to predict the next state, given the selected action. A policy $\pi(a \mid x)$ describes how the agent selects actions. To evaluate a decision, RL considers both its immediate reward and the rewards obtained thereafter. For a finite horizon of N decision steps, this quantity, called the return, is

$$G_t = \sum_{i=0}^{N-t-1} \gamma_{\mathrm{RL}}^i r(x_{t+i}, a_{t+i}),$$

and the learning objective is to find a policy that maximizes the expected return. The value function and action-value function of a policy π are defined as

$$v_t^\pi(x) = \mathbb{E}_\pi[G_t \mid X_t = x], \qquad q_t^\pi(x, a) = \mathbb{E}_\pi[G_t \mid X_t = x, A_t = a].$$

The value function estimates the expected return from a state, whereas the action-value function estimates the expected return from a state-action pair. These estimates allow the agent to consider the future consequences of a decision, not only its immediate reward.

For a finite-horizon problem, the optimal action-value function can be written recursively as

$$q_t^\star(x, a) = r(x, a) + \gamma_{\mathrm{RL}}\, \mathbb{E}\big[v_{t+1}^\star(x') \mid x, a\big].$$

Here, x' denotes the next state. Rather than calculating this expectation from a known system model, RL learns an approximation of the action-value function from sampled transitions (x_t, a_t, r_t, x_{t+1}). The policy is then improved to select actions with higher estimated long-term return. This illustrates the central idea of RL: actions are evaluated not only by their immediate reward, but also by their expected future consequences [7].

Classical dynamic programming can solve MDPs through backward recursion. However, this approach becomes impractical when the state or action space grows, a difficulty known as the curse of dimensionality [7]. Realistic energy systems often have continuous states, continuous actions, and long horizons, so exact dynamic-programming solutions are often computationally infeasible.

RL addresses this limitation by learning approximate policies from interaction data. A trained policy can then compute an action directly from the current observation. In model-free RL, the learning algorithm does not require an explicit analytical transition model for planning, although training still requires interaction

data. In engineering applications, this data is usually generated with a simulator or digital model, so the quality of the learned policy depends strongly on the quality of the training environment.

To locate the algorithm used in this thesis, it is helpful to briefly classify the main families of RL algorithms. RL methods are not separated by one single list of types; instead, they are usually described through several overlapping distinctions:

- Model-based and model-free methods: model-based methods learn or use a model of the environment dynamics for planning. Model-free methods learn directly from sampled transitions.
- Value-based and policy-based methods: value-based methods learn value or action-value functions and choose actions from them. Policy-based methods directly optimize a parameterized policy.
- Actor-critic methods: actor-critic methods combine ideas above. The actor represents the policy, and the critic estimates values or action-values used to improve that policy.
- On-policy and off-policy methods: on-policy methods learn from data generated by the current policy. Off-policy methods can also reuse data generated by previous policies or exploratory behavior.
- Tabular and deep RL methods: tabular methods store values explicitly for small state and action spaces. Deep RL uses neural networks as function approximators and is therefore more suitable for large or continuous spaces.

The relevant setpoints in this thesis, including battery power, temperature levels, mass flow rates, and bypass positions, are continuous quantities. Discretizing them would either exclude useful operating points or create a very large action space. Twin Delayed Deep Deterministic Policy Gradient (TD3) and Soft Actor Critic (SAC) are therefore selected because they learn parameterized policies that output continuous actions directly.

Both methods use an actor-critic architecture. The actor maps the current state to continuous setpoint decisions, while the critic estimates their expected long-term effect on operating cost and future system states. TD3 and SAC are applied to learn continuous setpoint policies for both case studies.

2.3.1 TD3 and SAC

Twin Delayed Deep Deterministic Policy Gradient (TD3). TD3 uses a deterministic actor, $\mu_\theta(x)$, that maps the current state directly to a continuous action. Two critics estimate the action value, and the smaller estimate is used when constructing the learning target to reduce overestimation bias. TD3 further stabilizes learning through delayed actor updates and target-policy smoothing, in which small noise is added to target actions during critic updates [10].

$$J_{\mathrm{TD3}}(\theta) = \mathbb{E}_{x\sim\mathcal{D}}\left[q_{\phi_1}\big(x, \mu_\theta(x)\big)\right].$$

Here, $J_{\mathrm{TD3}}(\theta)$ is the actor objective; θ denotes the actor parameters; $\mathcal{D}$ is the replay buffer from which states x are sampled; $\mu_\theta(x)$ is the deterministic action proposed by the actor; and q_{ϕ_1} is the first critic, parameterized by ϕ_1, which estimates the action value. The TD3 actor is trained to maximize this objective, producing one deterministic action, $a = \mu_\theta(x)$, for each state.

Soft Actor-Critic (SAC). SAC instead uses a stochastic actor, $\pi_\theta(a \mid x)$, that defines a distribution over continuous actions. In addition to maximizing expected return, SAC includes an entropy term that encourages exploration while training. Like TD3, SAC uses two critics and experience replay; however, its stochastic policy and entropy objective distinguish it from the deterministic TD3 policy [11].

$$J_{\mathrm{SAC}}(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{N-1} \gamma_{\mathrm{RL}}^t \left(r(x_t, a_t) + \alpha \mathcal{H}(\pi(\cdot \mid x_t)) \right) \right].$$

where $J_{\mathrm{SAC}}(\pi)$ is the objective of the stochastic policy π; $\mathbb{E}_\pi$ denotes the expectation over trajectories generated by that policy; N is the number of decision steps; γ_{RL} is the discount factor; $r(x_t, a_t)$ is the reward at time t; α weights the entropy term; and $\mathcal{H}(\pi(\cdot \mid x_t))$ is the policy entropy.

For a state x, the policy entropy is defined as

$$\mathcal{H}\big(\pi(\cdot \mid x)\big) = -\mathbb{E}_{a \sim \pi(\cdot|x)} \left[\log \pi(a \mid x) \right].$$

where a is an action sampled from the policy, and $\log \pi(a \mid x)$ is the logarithm of its probability density. During training, entropy is estimated from sampled actions as $\mathcal{H}(\pi(\cdot \mid x)) \approx -\log \pi(a \mid x)$ [11].

The two algorithms therefore provide a comparison between deterministic and stochastic actor–critic policies for continuous setpoint optimization. Their performance is evaluated using operating cost, constraint satisfaction, and computation time.

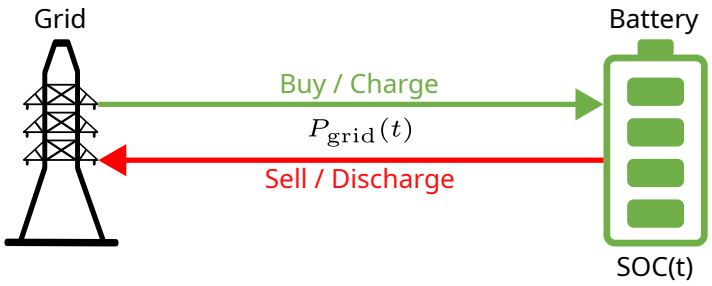


Figure 2: Schematic of the battery energy storage system. The battery charges from the grid and sells electricity back to the grid; its state of charge SOC(t) records the stored energy.

3 Use Cases

This section presents the two case studies used to evaluate reinforcement learning for setpoint determination at different levels of system complexity. The first subsection introduces the battery energy storage system as the simpler case, with one control command and state-of-charge dynamics. The second subsection presents the cookie production factory as the more complex case, with multiple controllable components, nonlinear process behavior, thermal energy storage, and coupled operational constraints.

3.1 Battery Energy Storage System

Battery energy storage systems (BESS) are electrochemical devices that store electrical energy for later use and provide grid services such as energy arbitrage, peak shaving, and short-term balancing. A typical BESS is characterized by its usable energy capacity, maximum charge/discharge power, round-trip efficiency, and state-of-charge (SOC) operating limits. Real-world operation must balance economic objectives (e.g., buying electricity at low prices and selling it at high prices) with engineering constraints (SOC bounds, power limits, efficiency losses).

BESS presents several challenges: Decisions are inherently time-coupled because current charging/discharging changes future availability. In addition, uncertain exogenous signals (market prices) require policies that are robust to forecast errors, and hard feasibility constraints must be enforced at every time step. These properties make BESS an illustrative case study for comparing optimization-based and reinforcement-learning approaches to real-time setpoint determination in energy systems.

A battery energy storage system provides flexible power by buying electricity when prices are low and selling it back to the grid when prices are high. This price-arbitrage operation maximizes profit. For simplicity, the same time-varying electricity price S_t is used for purchases and sales.

The SOC dynamics are modeled as

$$\text{SOC}_{t+1} = \text{SOC}_t + \frac{P_{\text{actual},t}\,\Delta t}{E_{\max}},$$

where $P_{\text{actual},t}$ is the effective storage-side power after efficiency losses, $E_{\max}$ the usable energy capacity, and Δt the time step. The sign convention is $P_t^{\text{S}} > 0$ for charging and grid purchase, and $P_t^{\text{S}} < 0$ for discharging and electricity sale. The superscript S identifies power exchanged with the electricity supply grid. The mathematical formulation uses the following smooth power and terminal constraints:

$$
\begin{aligned}
P_t^{\mathrm{S}} = P_{\max}\Big[&\frac{1}{2}\left(1+\tanh\big(k(u_t-\delta)\big)\right)u_t \\
&\times\frac{1}{2}\left(1+\tanh\big(k(0.995-\mathrm{SOC}_t)\big)\right) \\
&+\frac{1}{2}\left(1-\tanh\big(k(u_t+\delta)\big)\right)u_t \\
&\times\frac{1}{2}\left(1+\tanh\big(k(\mathrm{SOC}_t-0.005)\big)\right)\Big], \quad \forall t, \qquad (4)
\end{aligned}
$$

$$
\begin{aligned}
P_{\mathrm{actual},t} = \frac{P_t^{\mathrm{S}}}{2}\Bigg[&\left(1+\tanh\left(\frac{P_t^{\mathrm{S}}}{\epsilon}\right)\right)\eta \\
&+\left(1-\tanh\left(\frac{P_t^{\mathrm{S}}}{\epsilon}\right)\right)\frac{1}{\eta}\Bigg], \quad \forall t, \qquad (5)
\end{aligned}
$$

$$
\mathrm{SOC}_{\mathrm{final}} \geq \mathrm{SOC}_{\mathrm{term}}. \qquad (6)
$$

Here, u_t is the normalized control command and $P_{\max}$ is the maximum charging and discharging power. The parameter $k = 700$ controls the steepness of the smooth action and SOC gates: increasing k makes their transitions more abrupt, whereas decreasing it produces a wider and more gradual transition. The parameter δ shifts the charging and discharging gates away from zero and thereby defines a smooth deadband for small control commands. The implemented value $\delta = 0.0025$ corresponds to 0.25% of the maximum normalized action magnitude, or a nominal power command of 0.05 kW for $P_{\max} = 20\,\mathrm{kW}$. This small numerical deadband attenuates near-zero commands and reduces switching around zero while leaving the usable action range effectively unchanged.

The parameter η describes the charging and discharging efficiency. The power scale $\epsilon = 1\,\mathrm{kW}$ determines the width of the smooth transition between the charging and discharging efficiency relations in Eq. (5). A larger ϵ produces a more gradual transition around $P_t^{\mathrm{S}} = 0$, while a smaller value more closely approximates an abrupt change between the two operating directions. Equation (4) maps the normalized action and current SOC to grid-side battery power. It progressively restricts charging as SOC approaches 0.995 and discharging as SOC approaches 0.005. Equation (5) maps grid-side power to storage-side power through the smooth efficiency relation. Finally, the mathematical-optimization model enforces the terminal-SOC requirement $\mathrm{SOC}_{\mathrm{term}}^{\min} = 0.5$ in Eq. (6) as a hard constraint.

The battery model has a one-dimensional action space at each time step because the scalar command u_t is the only independently selected control variable. Over the complete horizon, the mathematical optimization therefore determines one charging or discharging command for each time step, while battery power and SOC follow from the model equations.

The battery objective is to maximize profit. The grid-side cost contribution is calculated as the electricity price multiplied by the battery power, and its minimization is equivalent to profit maximization. Accordingly, the mathematical-optimization objective is

$$
\min_{\{u_t\}_{t=0}^{N-1}} J = \sum_{t=0}^{N-1} S_t P_t^{\mathrm{S}} \Delta t,
$$

where S_t is the electricity price at time step t. The battery purchases electricity from the grid when prices are low and sells it back to the grid when prices are high.

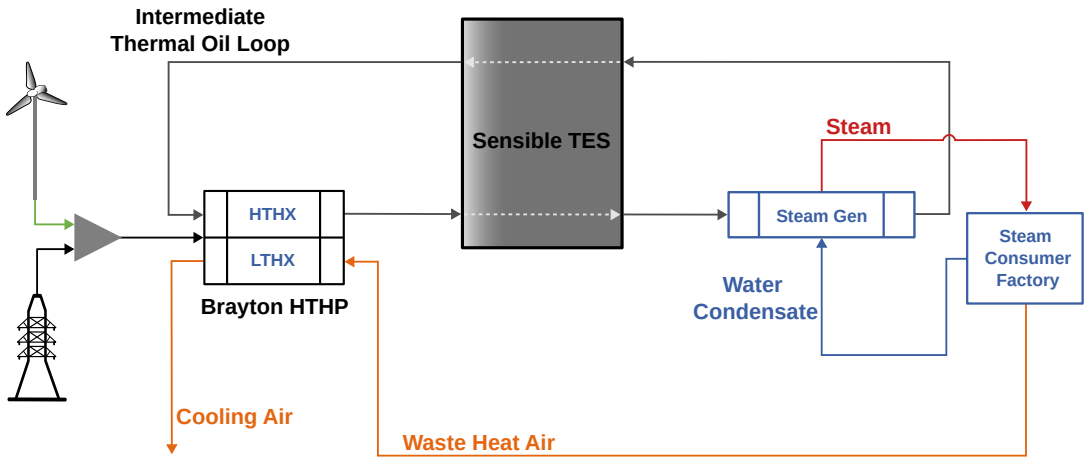


Figure 3: Schematic representation of the electrified steam-generation system serving the cookie production factory. Reproduced from Kyriakidis et al. [9]; the system was originally introduced by Walden et al. [3].

Main challenges include respecting SOC and power limits, accounting for efficiency losses, handling uncertain future prices, and managing the fact that actions now affect future energy availability. These features make the BESS a suitable test case for reinforcement learning, where a policy must balance using energy now against saving it for likely higher-price periods while always obeying feasibility limits.

3.2 Cookie Production Factory

The second use case represents a non-linear cookie production factory with an electrified steam-generation utility system. The production process requires a constant supply of process heat in the form of superheated steam, while the utility system must decide how to operate the energy conversion components under fluctuating wind generation W_t and electricity prices S_t. The operational objective is to satisfy the steam demand while minimizing grid electricity cost over a finite scheduling horizon. The system model is based on the industrial power-to-heat system introduced by Walden et al. [3] and later used as a renewable steam-generation benchmark by Kyriakidis et al. [9].

The system consists of four main units: a wind turbine (WT), a closed reverse Brayton-cycle high-temperature heat pump (HTHP), a sensible thermal energy storage (TES), and a steam generator (SG). The WT supplies renewable electricity, while additional electricity can be purchased from the grid. The HTHP uses a waste-heat air stream as its heat source and delivers high-temperature process heat to an intermediate thermal-oil loop. The thermal oil acts as heat-transfer fluid and can be routed through the TES or directly to the SG by controllable bypasses. The SG provides the required steam conditions for the consumer factory. The HTHP and SG are represented by polynomial surrogate models, while the TES is modeled as a simplified single energy reservoir, and the charging/discharging behavior is represented using efficiency equations.

The corresponding finite-horizon optimization problem is formulated over a dis-

crete time horizon with time index $t \in \{1, \ldots, n\}$:

$$\min_{x} \quad J = \sum_{t=1}^{n} P_t^{\mathrm{S}} S_t \Delta t, \tag{7}$$

$$\text{s.t.} \quad P_t^{\mathrm{S}} + W_t = 3F_{\mathrm{HTHP}}\left(T_{\mathrm{I}}^t, \dot{m}_{\mathrm{I}}^t, T_{\mathrm{III}}^t, R^t\right), \tag{8}$$

$$T_{\mathrm{II}}^t = F_{\mathrm{HTHX}}\left(T_{\mathrm{I}}^t, \dot{m}_{\mathrm{I}}^t, T_{\mathrm{III}}^t, R^t\right), \tag{9}$$

$$T_{\mathrm{IV}}^t = F_{\mathrm{LTHX}}\left(T_{\mathrm{I}}^t, \dot{m}_{\mathrm{I}}^t, T_{\mathrm{III}}^t, R^t\right), \tag{10}$$

$$T_2^t = \beta_1^t T_{\mathrm{II}}^t + \left(1 - \beta_1^t\right) T_1^t, \tag{11}$$

$$T_{\mathrm{I}}^t = \beta_2^t T_3^t + \left(1 - \beta_2^t\right) T_4^t, \tag{12}$$

$$T_2^t = 201.92 + \frac{1819.32}{3\dot{m}_{\mathrm{II}}^t}, \tag{13}$$

$$T_3^t = 196.3 - \frac{188.4}{3\dot{m}_{\mathrm{I}}^t}, \tag{14}$$

$$\dot{Q}_{s,\mathrm{ch}}^t = 3\dot{m}_{\mathrm{II}}^t c_{p,\mathrm{f}}\left(T_{\mathrm{II}}^t - T_1^t\right)\left(1 - \beta_1^t\right), \tag{15}$$

$$\dot{Q}_{s,\mathrm{dch}}^t = 3\dot{m}_{\mathrm{I}}^t c_{p,\mathrm{f}}\left(T_4^t - T_3^t\right)\left(1 - \beta_2^t\right), \tag{16}$$

$$\dot{Q}_{s,\mathrm{ch}}^t \dot{Q}_{s,\mathrm{dch}}^t \leq \gamma, \tag{17}$$

$$T_1^t = T_{\mathrm{II}}^t - \epsilon_{\mathrm{ch}}\left(T_{\mathrm{II}}^t - T_s^{t-1}\right), \tag{18}$$

$$T_4^t = T_3^t - \epsilon_{\mathrm{dch}}\left(T_3^t - T_s^{t-1}\right), \tag{19}$$

$$T_s^t = T_s^{t-1} + \frac{\dot{Q}_{s,\mathrm{ch}}^t - \dot{Q}_{s,\mathrm{dch}}^t}{m_s c_{p,\mathrm{s}}} \Delta t, \tag{20}$$

$$T_s^0 = \tilde{T}_0, \qquad T_s^n = T_s^0, \tag{21}$$

$$T_{\mathrm{I}}^t \in [177, 250], \qquad \dot{m}_{\mathrm{I}}^t \in [5, 16], \tag{22}$$

$$T_{\mathrm{III}}^t \in [60, 100], \qquad R^t \in [0.8, 1.53], \qquad \beta_1^t, \beta_2^t \in [0, 1]. \tag{23}$$

Here, S_t denotes the spot electricity price, P_t^{S} denotes grid power, and W_t denotes available wind power at time step t. The objective in Eq. (7) minimizes the electricity cost of grid power consumption. The power balance in Eq. (8) couples grid electricity, wind power, and HTHP demand. Equations (8)–(23) comprise 16 numbered constraint relations. Together, they describe the HTHP and SG surrogate models, bypass mixing, TES charging and discharging, the prohibition of simultaneous charge and discharge, cyclic storage operation, and the admissible operating ranges of the main decision variables.

The cookie-factory mathematical-optimization model has a five-dimensional action space at each time step. Its independently selected operating variables are the two bypass variables β_1^t and β_2^t, the compressor-speed-related variable R^t, the mass flow rate $\dot{m}_{\mathrm{I}}^t$, and the HTHP inlet temperature T_{I}^t. The remaining temperatures, power flows, and storage quantities are coupled through the model equations. In the RL formulation, the action space is reduced to two dimensions: the policy selects a signed storage-mode command and the mass flow rate $\dot{m}_{\mathrm{I}}^t$. The environment maps the storage-mode command to β_1^t and β_2^t and derives R^t from the F_4 coupling equation. This use case is more complex than the battery system because it combines non-linear surrogate equations, thermal storage dynamics, and hard process constraints. It therefore provides a suitable second benchmark for evaluating whether reinforcement learning can learn feasible and cost-effective setpoint policies for industrial energy utility systems, and for comparing the learned policy against the existing mathematical optimization formulation.

4 Methodology

The previous section introduced the two energy utility systems considered in this thesis and described their operational objectives and constraints. This section explains how these system-specific optimization problems are reformulated as reinforcement-learning problems. In each time step, the agent observes the current system state together with current and forecast external conditions, such as electricity prices and available wind power, selects continuous control actions, and receives a reward based on economic performance and constraint satisfaction. Although the physical models and control variables differ between the two use cases, both are represented using the same agent-environment interaction structure. At each discrete time step, the environment maps the agent's action to physical setpoints, advances the system dynamics, and returns the resulting next state, reward, and termination signals.

4.1 Problem Formulation

For both use cases, the objective is to determine operating setpoints over a discrete time horizon that optimize the relevant economic objective while respecting all constraints. Let $t \in \{0, \dots, N-1\}$ denote the discrete time index, S_t the spot electricity price, P_t^{S} the grid power, and W_t the available wind power. At each time step, the system is described by a state vector x_t, the agent selects a continuous action vector a_t, and the system evolves to the next state x_{t+1} according to the underlying system dynamics. For the battery case, the only external input is the electricity price S_t. For the cookie-factory case, the external inputs are the electricity price S_t and the available wind power W_t.

The following subsections specify the state variables, action spaces, transition dynamics, and reward functions for each system.

The reward definitions follow the respective operational objectives and include penalty terms for constraint violations where required. In both cases, feasible and economically favorable operation corresponds to a high cumulative reward. The following subsections describe how this general formulation is implemented in the reinforcement-learning framework, training setup, and evaluation procedure.

4.2 Reinforcement Learning Framework

The reinforcement-learning framework used in this thesis follows the same general structure for both use cases. The environments are implemented using Gymnasium [18], while the continuous-control policies are trained using Stable-Baselines3 [19]. Each interaction is stored as a transition

$$\tau_t = (x_t, a_t, r_t, x_{t+1}, d_t)\,,$$

where d_t indicates whether the episode has ended. In the present environments, reaching the end of the fixed scheduling horizon is represented by the `terminated` signal. The transitions are stored in a replay buffer and sampled repeatedly during training, enabling the off-policy learning used by SAC and TD3. This implementation follows the general agent–environment interaction illustrated in Figure 1, extended by forecast-augmented states, environment-specific action mappings, and constraint handling.

During training, the policy action is first processed by an environment-specific action map

$$a_t^{\text{phys}} = \mathcal{M}(x_t, a_t),$$

where $\mathcal{M}$ maps the normalized policy output to physical setpoints and applies environment-specific feasibility handling. Hard constraints are enforced through action bounds, state-dependent mappings, or the system dynamics, while requirements that are not enforced directly are represented by penalty terms in the reward.

For the battery system, the state supplied to the agent consists of the current state of charge, the normalized current spot electricity price S_t, the normalized position within the scheduling horizon, and the normalized 24-step price forecast $S_{0:N-1}$. As future work, a rolling-horizon implementation could update the forecast at every decision step. Here, the complete forecast is kept fixed to enable a direct comparison with mathematical optimization over the same single scheduling horizon. The continuous scalar action $u_t \in [-1, 1]$ represents the battery charging or discharging command: positive values correspond to charging and negative values to discharging. A smooth, state-dependent action map converts this normalized command into grid-side battery power while progressively restricting charging and discharging near the upper and lower SOC limits.

The reward has the same general form in both environments:

$$r_t = -C_t - \sum_{\ell} \lambda_\ell v_{\ell,t} - \lambda_{\text{T}} v_{\text{T},t},$$

where C_t is the grid-electricity cost at time step t, $v_{\ell,t}$ are constraint-violation terms, and λ_ℓ are their corresponding penalty weights. The terminal term $v_{\text{T},t}$ is only active at the end of an episode, and λ_{T} is its penalty weight. In the battery case it penalizes an insufficient terminal SOC, while in the cookie factory it penalizes deviation from the cyclic terminal TES temperature. This reward design converts the case-specific economic objective into a return-maximization problem without removing the need for physical feasibility.

More specifically, the battery reward implemented in the environment is

$$r_t = -\frac{S_t P_t^{\text{S}} \Delta t}{1000} - \mathbf{1}_{\{t=N-1\}} \lambda_{\text{SOC}} \max\left(SOC_{\min}^{\text{terminal}} - SOC_N, 0\right),$$

where S_t is the spot electricity price in EUR/MWh, P_t^{S} is the grid-side battery power in kW, and Δt is the step duration in hours. The factor 1000 converts the energy charge to euros. The terminal term is applied only at the end of an episode and penalizes a final SOC below the prescribed minimum. Intermediate SOC feasibility is handled directly by the battery dynamics and state-dependent power mapping rather than by a constraint-violation penalty.

As an additional feasibility measure, a progressive terminal-SOC reachability penalty was investigated. At each time step, the environment calculates the minimum SOC required to retain a feasible path to the terminal-SOC target under the maximum possible future charging rate:

$$\text{SOC}_t^{\text{reach}} = \max\left(\text{SOC}_{\min}^{\text{terminal}} - (N-t)\frac{\eta P_{\max} \Delta t}{E_{\max}}, 0\right).$$

Whenever the current SOC falls below this reachability threshold, the shaped reward is

$$r_t^{\text{reach}} = r_t - \lambda_{\text{reach}} \max\left(\text{SOC}_t^{\text{reach}} - \text{SOC}_t, 0\right),$$

where $\lambda_{\text{reach}} > 0$ weights the reachability penalty. Unlike the terminal penalty described above, this penalty provides feedback before the final time step and encourages the policy to preserve sufficient energy for the terminal requirement. The resulting feasibility–cost trade-off is reported in Section 5.

After training, both policies are evaluated deterministically to compare schedules. For SAC, the mean action of the stochastic policy is selected at each time step. TD3 directly outputs a deterministic action. This produces one reproducible setpoint trajectory for a given price and wind profile and allows direct comparison with the mathematical-optimization baseline in terms of cost, constraint satisfaction, and computation time.

4.3 Training Setup

Training is performed in reinforcement-learning environments before the learned policies are evaluated on fixed test trajectories. The environment provides the transition dynamics, cost calculation, constraint handling, and terminal condition for each episode. The learning algorithm only receives observations, rewards, and termination signals through the environment interface.

For both use cases, one episode corresponds to a 24-step scheduling horizon. During training, the start index of the episode is sampled randomly from the available time-series data. This random-window strategy exposes the agent to different price and renewable-generation patterns instead of repeatedly starting each episode at the same time of day. Both environments use the grid-price series; the cookie factory environment additionally samples the corresponding wind-availability values for the same 24-hour window.

The training environments are wrapped with monitoring and vectorized environment interfaces. Monitoring records episode rewards and lengths for later inspection, while the vectorized wrapper provides the interface expected by the Stable-Baselines3 implementation of SAC and TD3. Only one environment instance is used at a time in the current setup, so the vectorization is mainly an interface layer rather than a parallel simulation strategy.

The BESS reward is the negative grid-side electricity cost at each time step, with an additional terminal penalty if the final SOC falls below the desired terminal level. The cookie factory reward also starts from the negative grid-electricity cost, but includes additional penalties for different constraint violations. The penalty terms are therefore more important in the second use case, because feasibility cannot be represented by simple action bounds alone.

The main SAC and TD3 hyperparameters used in the implementation are summarized in Table 2. The same learning-rate schedule and discount factor are used for both use cases to provide a consistent baseline configuration. The learning-rate schedule was selected through preliminary experiments. Differences between the two training setups mainly arise from the different state variables, action spaces, and constraint structures of the two environments.

Both SAC policies use three hidden layers of 256 neurons with ReLU activation functions for both actor and critic networks. TD3 uses the Stable-Baselines3 default architecture of two ReLU hidden layers with 400 and 300 neurons for both actor and critic networks. In all cases, trained models are saved after learning and later loaded for evaluation.

Parameter	Battery system	Cookie factory
Algorithm	SAC and TD3	SAC and TD3
Policy	MLP policy	MLP policy
Network architecture	SAC:3×256; TD3:400,300 ReLU	SAC:3×256; TD3:400,300 ReLU
Replay buffer size	200000	200000
Batch size	256	256
Learning rate	linear schedule from $3 \cdot 10^{-4}$	linear schedule from $3 \cdot 10^{-4}$
Discount factor γ_{RL}	0.99	0.99
Target smoothing coefficient τ	0.005	0.005
Episode length	24 steps	24 steps

Table 2: Main SAC and TD3 training settings used for the two reinforcement-learning environments. TD3 uses the Stable-Baselines3 default network architecture; the remaining listed settings are shared.

4.4 Evaluation Metrics

The trained RL policies are evaluated through deterministic rollouts on fixed 24-hour evaluation windows. This makes the resulting setpoint trajectory reproducible and allows a direct comparison with the mathematical optimization baseline on the same grid-price and wind-availability profiles.

The primary economic metric is the total net grid-side cost over the evaluation horizon:

$$J_{\mathrm{grid}} = \sum_{t=0}^{N-1} C_t,$$

where C_t is the grid-electricity cost at time step t. For the battery system, this cost is computed from the electricity price and the battery power exchanged with the grid, using the same price for purchases and sales; minimizing J_{grid} is equivalent to maximizing profit. For the cookie factory, it is computed from the grid power required by the heat-pump system after accounting for available wind power and is minimized directly. Thus, a lower value of J_{grid} indicates higher profit for the battery and lower operating cost for the cookie factory.

The cumulative reward is reported as a secondary metric:

$$R_{\mathrm{ep}} = \sum_{t=0}^{N-1} r_t.$$

This value is useful for analyzing the RL objective because it combines the economic term with penalty terms. However, it is not used as the only comparison criterion, since the reward may include artificial penalty weights that are not part of the physical economic objective. Therefore, the economic objective and constraint satisfaction are evaluated separately.

Constraint satisfaction is measured by aggregating the violation terms recorded during the rollout. For a set of constraint residuals or violation magnitudes $v_{\ell,t}$, the total violation is defined as

$$V_{\mathrm{tot}} = \sum_{t=0}^{N-1} \sum_{\ell} v_{\ell,t}.$$

In addition, the number or frequency of time steps with nonzero violations is considered:

$$f_{\text{viol}} = \frac{1}{N} \sum_{t=0}^{N-1} \mathbb{I} \left(\sum_{\ell} v_{\ell,t} > \varepsilon \right),$$

where ε is a small numerical tolerance. For the battery system, the relevant checks include SOC bounds, power limits, and the terminal SOC requirement. For the cookie factory, the violation categories include bypass feasibility, complementarity of charging and discharging, TES temperature bounds, grid-power bounds, coupling residuals, and the terminal cyclic TES condition.

The evaluation also records trajectory-level operating indicators. For the BESS, these include the SOC trajectory, commanded and effective battery power, and terminal SOC.

Finally, computation time is used to assess practical applicability. For mathematical optimization, the relevant value is the solver time required to compute a complete schedule for the evaluation horizon. For RL, the one-time training effort is reported separately from the online evaluation time. This distinction is important because RL can require substantial training effort, but a trained policy can usually produce setpoints much faster than solving a nonlinear optimization problem from scratch for every horizon.

Metric	SAC policy	TD3 policy	Mathematical optimization
Mean objective [EUR]	-2.8911 ± 1.2408	-3.0223 ± 1.2208	-3.1851 ± 1.2212
Objective range [EUR]	$[-7.7304, -0.3483]$	$[-7.7841, -0.2604]$	$[-7.8530, -0.4353]$
Mean computation time [s]	0.0126 ± 0.0105	0.0082 ± 0.0067	0.5605 ± 0.6295
Time range [s]	$[0.0102, 0.3444]$	$[0.0070, 0.2150]$	$[0.1164, 9.2371]$
Mean maximum SOC	0.8661 ± 0.1620	0.8905 ± 0.1458	0.8793 ± 0.1595
Mean minimum SOC	0.2129 ± 0.1887	0.1947 ± 0.1776	0.1925 ± 0.1796

Table 3: Comparison between the trained SAC and TD3 policies and mathematical optimization over 1000 random 24-hour windows. Mathematical optimization serves as the baseline; lower (more negative) objective values correspond to higher profit.

5 Results and Discussion

The previous section described the RL formulation, training setup, and evaluation procedure for both use cases. This section presents and discusses the resulting policy performance relative to mathematical optimization. All computations were performed in Python 3.14.1, with Gymnasium 1.2.2 used to implement the reinforcement-learning environments and Stable-Baselines3 2.7.0, based on PyTorch 2.9.1, used to train the RL policies. Experiments were conducted on an AMD Ryzen 7 7840HS CPU with Radeon 780M Graphics.

5.1 Battery Energy Storage System

The battery case study was first used to compare different learning-rate schedules for SAC and TD3. Each configuration was trained and evaluated over 50 runs on the same battery environment and evaluation setup.

The linear schedule achieved the best mean reward and the lowest mean objective value among the tested configurations. However, the differences between the three schedules were small relative to the variation in objective values across runs. This indicates that the battery environment is not highly sensitive to the exact learning-rate decay shape in this experiment. The linear schedule was therefore used as a stable baseline for the subsequent comparison against mathematical optimization.

The trained SAC and TD3 policies were compared with mathematical optimization over 1000 randomly sampled 24-hour price windows. Table 3 summarizes the resulting objective values, computation times, and SOC ranges, using mathematical optimization as the baseline.

Mathematical optimization achieved the lowest mean objective value and therefore the highest mean profit. It was the best-performing method overall. Among SAC and TD3, TD3 performed better. It achieved a lower mean objective value and higher mean profit, and was closer to the mathematical-optimization baseline. The mean objective gap was 0.1628 ± 0.1284 EUR for TD3 and 0.2215 ± 0.0811 EUR for SAC. Since the objective is minimized, both positive gaps indicate that the RL policies achieved lower profit than mathematical optimization. TD3 performed better in this relatively simple and deterministic environment. This could be explained by TD3 directly maximizing the discounted return, whereas SAC balances discounted-return maximization with maintaining policy entropy. Further TD3 results are presented below.

The small objective gaps shown in Figure 4 indicate that the TD3 policy closely approaches the mathematical-optimization solution across the selected comparison windows.

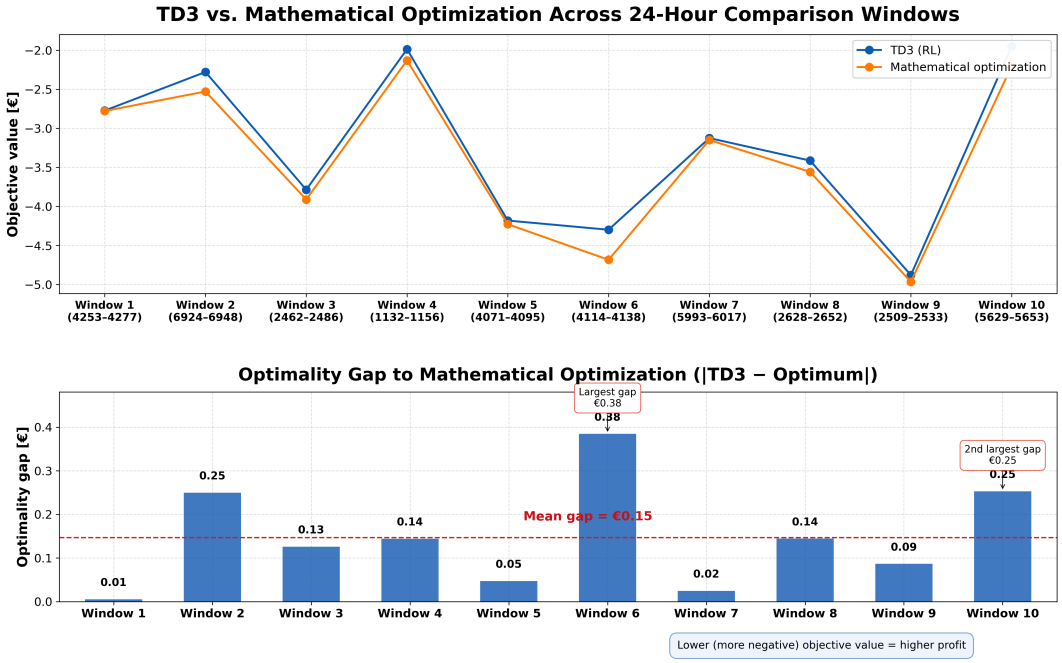


Figure 4: TD3 and mathematical-optimization objective values for ten randomly selected 24-hour comparison windows. The lower panel shows the corresponding TD3–mathematical-optimization gaps.

The SOC trajectories are also similar in the current comparison. The TD3 policy reached a mean minimum SOC of 0.1947 and a mean maximum SOC of 0.8905, while the mathematical solution reached a mean minimum SOC of 0.1925 and a mean maximum SOC of 0.8793. This indicates that both methods used a comparable portion of the available storage flexibility.

Figure 5 shows an illustrative 24-hour dispatch trajectory for a TD3 policy and the mathematical optimization solution over the 24-hour horizon. The upper panel compares grid-power profiles and electricity price, showing that the TD3 actions closely follow the mathematical-optimization schedule. The lower panel compares SOC trajectories, both of which satisfy the terminal-SOC requirement.

In a separate feasibility evaluation, the TD3 policy without the progressive reachability penalty violated the terminal-SOC constraint in 4% of the evaluation windows. Adding the progressive penalty reduced the violation rate to zero but increased the mean objective gap between TD3 and mathematical optimization to 0.2436 ± 0.1473 EUR, illustrating the trade-off between strict terminal-constraint satisfaction and economic performance. Upon further investigation, the mean terminal-SOC shortfall without the progressive penalty was only 0.000252. Given its higher profit, this small shortfall was considered tolerable, and the TD3 policy without the progressive reachability penalty was selected as the final model.

The main advantage of the TD3 policy was computation time. The trained TD3 model required, on average, 0.0082 s to generate a 24-hour rollout, while mathematical optimization required 0.5605 s on average to solve the corresponding optimization problem. This corresponds to an average speedup of approximately 68.13×. For TD3, approximately one hour of offline training is required beforehand and is therefore not included in the online computation time. These results therefore illustrate the central trade-off observed in the battery case: RL provides very fast online setpoint generation after training and approaches the mathematical optimization

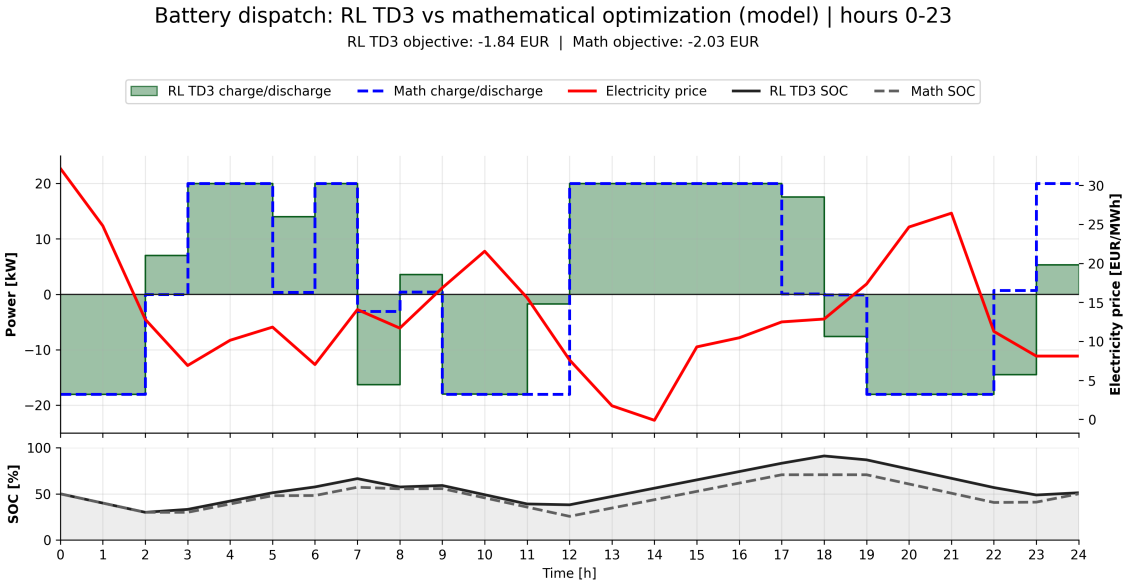


Figure 5: Illustrative 24-hour battery dispatch comparison between the TD3 policy and mathematical optimization.

result closely, while direct optimization still provides the best objective value.

5.2 Cookie Production Factory

The trained SAC policy was compared with mathematical optimization over ten matched 24-hour windows. Table 4 summarizes the economic objective, computation time, and aggregate constraint violation. The mathematical-optimization formulation achieved the lower cost in all ten windows. Its mean cost was 1898.6259 ± 694.5588 EUR, compared with 2019.9256 ± 637.5719 EUR for SAC. The resulting mean SAC–mathematical-optimization gap was 121.2997 EUR, and the mean absolute relative gap was 12.94%.

The trained SAC policy required 0.0455 s on average to generate a schedule, whereas mathematical optimization required 14.4743 s. Thus, SAC was approximately 318.39× faster during online evaluation; its offline training time is not included in this comparison. This reduction in computation time was accompanied by lower economic performance and weaker constraint satisfaction. The mean total constraint violation was 1.4341 for SAC, while mathematical optimization produced no constraint violation within numerical solver tolerance. For SAC, the nonzero mean contributions were 0.3867 from the F_4 coupling residual and 1.0473 from the terminal cyclic-storage condition; the reported flow, grid-power, inlet-temperature, and state-bound violations were zero on average.

Figure 6 illustrates the schedules produced for one 24-hour window. In this

Metric	SAC policy	Mathematical optimization
Mean cost [EUR]	2019.9256 ± 637.5719	1898.6259 ± 694.5588
Mean computation time [s]	0.0455	14.4743
Mean total constraint violation	1.4341	None within numerical tolerance

Table 4: Comparison of the trained SAC policy and mathematical optimization over ten matched 24-hour cookie-factory windows. Lower values are better for the cost and constraint-violation metrics.

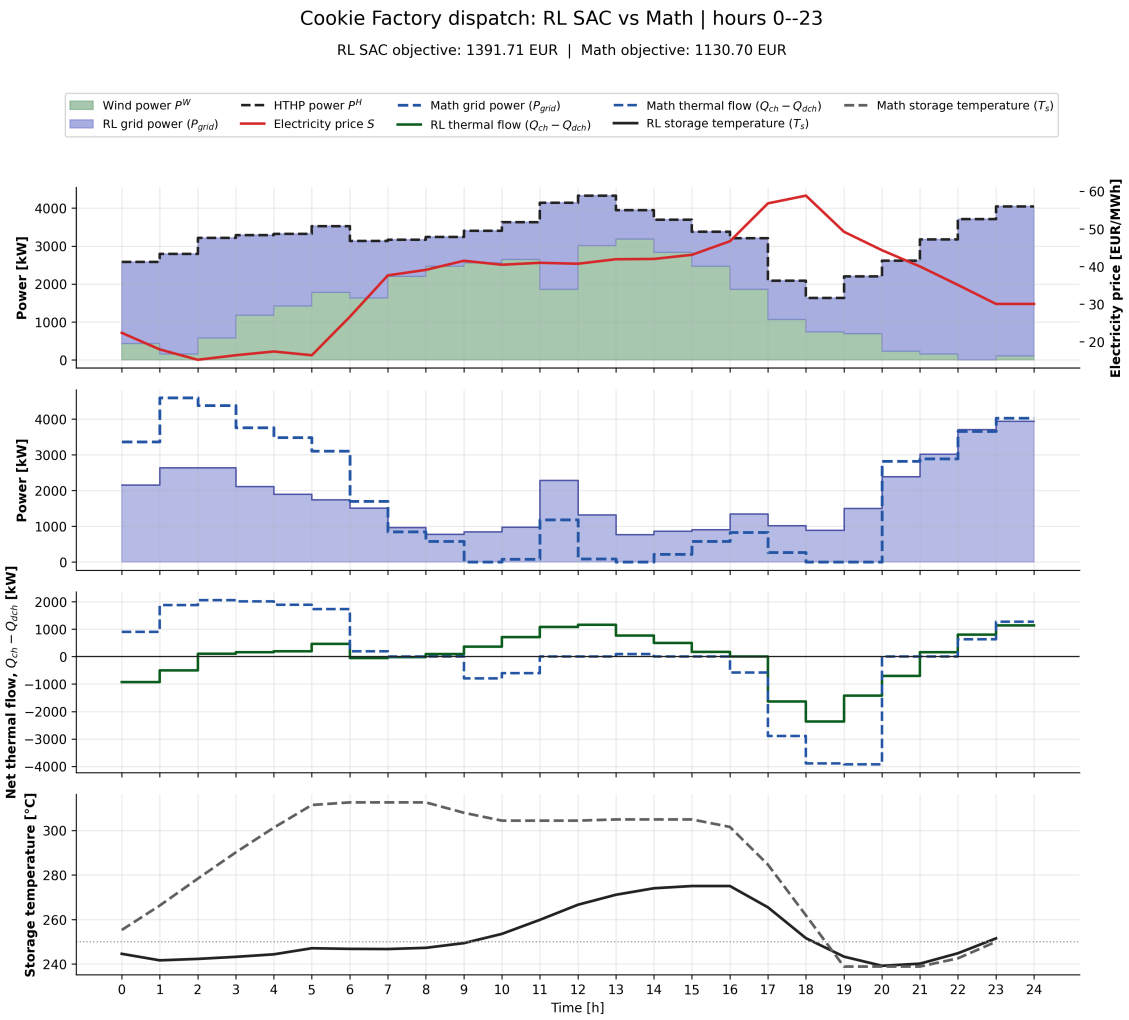


Figure 6: Illustrative 24-hour cookie-factory dispatch comparison between the SAC policy and mathematical optimization. "Math" denotes mathematical optimization. Positive net thermal flow indicates TES charging, and negative net thermal flow indicates discharging.

example, SAC obtained a cost of 1391.71 EUR, while mathematical optimization obtained 1130.70 EUR, corresponding to a gap of 261.01 EUR, or approximately 23.1% relative to the mathematical-optimization result. Both schedules respond to the available wind power and time-varying electricity price. However, mathematical optimization uses a broader TES temperature range: it charges the storage during the earlier part of the horizon and discharges strongly during the high-price period around hours 17–20. The SAC policy follows a less pronounced storage trajectory and consequently requires more grid electricity in parts of the horizon.

These results show that the trained policy can generate setpoints rapidly using the reduced two-dimensional cookie-factory action space, but the mathematical-optimization baseline retains an advantage in both operating cost and feasibility.

5.3 Key Findings

6 Conclusion and Future Work

6.1 Conclusion

This thesis investigated whether reinforcement learning can determine feasible and cost-effective operating setpoints for energy utility systems of different complexity. The setpoint-selection tasks were formulated as finite-horizon Markov decision processes and evaluated for a battery energy storage system and the electrified steam-generation utility system of a cookie production factory. The battery represents a one-dimensional storage-control problem, whereas the cookie-factory mathematical-optimization model contains five coupled control variables and nonlinear process constraints. For the cookie-factory RL formulation, the action space was reduced to two dimensions by allowing the policy to select the storage mode and mass flow while deriving the remaining quantities through the process equations. In both cases, the trained policies were compared with mathematical optimization using the original economic objective, constraint satisfaction, and online computation time.

For the battery system, mathematical optimization achieved the highest mean profit, but the TD3 policy approached its performance closely. Across 1000 randomly sampled 24-hour windows, the mean objective was -3.0223 EUR for TD3 and -3.1851 EUR for mathematical optimization, giving a mean gap of 0.1628 EUR. TD3 also produced SOC trajectories that used a similar portion of the available storage range. Its main advantage was speed: a complete policy rollout required 0.0082 s on average, compared with 0.5605 s for mathematical optimization, corresponding to a speedup of approximately $68.13\times$. The terminal-SOC study showed that feasibility and economic performance must be considered together. A progressive reachability penalty eliminated the observed terminal violations, but it also increased the objective gap. Without this penalty, the violation rate was 4%, although the mean shortfall was only 0.000252.

The cookie factory presented a more difficult control problem because nonlinear surrogate equations, storage dynamics, and process constraints must be satisfied simultaneously. Across ten matched 24-hour windows, SAC achieved a mean cost of 2019.9256 ± 637.5719 EUR, while mathematical optimization achieved 1898.6259 ± 694.5588 EUR. The mean cost gap was 121.2997 EUR, and the mean absolute relative gap was 12.94%. SAC generated a schedule in 0.0455 s on average, compared with 14.4743 s for mathematical optimization, resulting in a speedup of approximately $318.39\times$. However, the SAC schedules retained violations of the F_4 coupling relation and the cyclic terminal-storage condition, while mathematical optimization produced no constraint violation within numerical solver tolerance. Reducing the policy action space to two dimensions simplified the learning problem, but it did not by itself ensure satisfaction of all coupled constraints.

The results therefore provide a qualified answer to the research question. Reinforcement learning can learn continuous setpoint policies that produce economically meaningful schedules with very low online computation time. Its practical value is strongest when decisions must be generated repeatedly under short computation-time limits and when offline training can be performed in advance. In the simpler battery system, the learned policy remained close to the mathematical-optimization benchmark. As system complexity and constraint coupling increased, the gap in operating cost and feasibility became more pronounced. Mathematical optimization remained superior when the best objective value and reliable constraint satisfaction were the primary requirements.

6.2 Future Work

Constraint handling should be strengthened for the cookie-factory policy. The observed F_4 coupling residual and terminal cyclic-storage deviation could be addressed through constraint-preserving action mappings, action projection, constrained RL methods, or a small repair optimization applied to the policy output. A progressive reachability mechanism similar to the one studied for the battery could provide earlier feedback on whether the terminal TES target remains attainable. The two-dimensional action formulation should also be investigated further to determine how the derived compressor-speed calculation and storage-mode mapping affect feasibility, sensitivity, and learning stability.

The present evaluation uses fixed 24-hour price and wind profiles that are available to both methods. A rolling-horizon implementation should instead update forecasts and setpoints as new measurements become available. This would allow the policies to be tested under forecast errors, measurement noise, disturbances, and model–plant mismatch. Domain randomization or targeted retraining could then be used to improve robustness to operating conditions not represented in the original training data.

The mathematical-optimization comparison can also be extended. Warm-started IPOPT and hybrid methods such as BO-IPOPT should be evaluated on the same windows and computation budgets. A fair practical comparison should include solution quality, feasibility, initialization sensitivity, and computation time, while also accounting for the offline training cost of RL and the number of repeated online decisions required to amortize that cost. Hybrid approaches are particularly promising: a learned policy could provide a rapid initial schedule, followed by mathematical optimization to restore feasibility and improve the objective. Finally, the proposed policies should be validated using a higher-fidelity process model and, subsequently, a hardware-in-the-loop or real-plant implementation.

References

[1] M. Khalid. "Smart grids and renewable energy systems: Perspectives and grid integration challenges". In: *Energy Strategy Reviews* 51 (2024), p. 101299. DOI: 10.1016/j.esr.2024.101299.

[2] A. W"achter and L. T. Biegler. "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming". In: *Mathematical Programming* 106 (2006), pp. 25–57. DOI: 10.1007/s10107-004-0559-y.

[3] J. V. Walden et al. "Nonlinear operational optimization of an industrial power-to-heat system with a high temperature heat pump, a thermal energy storage and wind energy". In: *Applied Energy* 344 (2023), p. 121247. DOI: 10.1016/j.apenergy.2023.121247.

[4] M.-H. Lin, J.-F. Tsai, and C.-S. Yu. "A Review of Deterministic Optimization Methods in Engineering and Management". In: *Mathematical Problems in Engineering* 2012 (2012), p. 756023. DOI: 10.1155/2012/756023.

[5] A. Zhigljavsky and A. Žilinskas. *Stochastic Global Optimization*. Vol. 9. Springer Optimization and Its Applications. New York, NY: Springer, 2008. ISBN: 978-0-387-74022-5. DOI: 10.1007/978-0-387-74740-8.

[6] L. Kyriakidis, R. Kansara, and M. I. Roldán Serrano. "Energy management of industrial energy systems via rolling horizon and hybrid optimization: A real-plant application in Germany". In: *Energies* 18.15 (2025), p. 3977. DOI: 10.3390/en18153977.

[7] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018. URL: http://incompleteideas.net/book/the-book-2nd.html.

[8] E. Pilling, M. Bähr, and R. Wunderlich. *Reinforcement Learning Methods for the Stochastic Optimal Control of an Industrial Power-to-Heat System*. Dec. 2025. DOI: 10.48550/arXiv.2411.02211. URL: http://arxiv.org/abs/2411.02211 (visited on 02/11/2026).

[9] L. Kyriakidis, M. A. Mendez, and M. B"ahr. "A hybrid algorithm based on Bayesian optimization and Interior Point OPTimizer for optimal operation of energy conversion systems". In: *Energy* 312 (2024), p. 133416. DOI: 10.1016/j.energy.2024.133416. URL: https://linkinghub.elsevier.com/retrieve/pii/S036054422403192X.

[10] S. Fujimoto, H. van Hoof, and D. Meger. "Addressing function approximation error in actor-critic methods". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018, pp. 1587–1596. URL: https://proceedings.mlr.press/v80/fujimoto18a.html.

[11] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018, pp. 1861–1870.

[12] A. P. Tran and F. C. Yücel. "Safe and efficient control of a Brayton cycle heat pump using reinforcement learning". In: *Proceedings of the 16th International Modelica&FMI Conference*. Vol. 218. Linköping University Electronic Press, Oct. 2025, pp. 1017–1026. DOI: 10.3384/ecp2181017. URL: https://ecp.ep.liu.se/index.php/modelica/article/view/1389.

[13] P. H. Takam and N. Fruiba. *Stochastic optimal control problems for the cost-optimal management of a standalone microgrid*. Oct. 2025. DOI: 10.48550/arXiv.2511.00167. URL: https://arxiv.org/abs/2511.00167.

[14] Q. Meng, S. Hussain, F. Luo, Z. Wang, and X. Jin. "An online reinforcement learning-based energy management strategy for microgrids with centralized control". In: *IEEE Transactions on Industry Applications* 61.1 (2025), pp. 1501–1510. DOI: 10.1109/TIA.2024.3430264.

[15] J. Nocedal and S. J. Wright. *Numerical Optimization*. 2nd ed. New York: Springer, 2006. DOI: 10.1007/978-0-387-40065-5.

[16] L. Kyriakidis, M. Bähr, and M. A. Mendez. "Enhanced hybrid algorithm based on Bayesian optimization and Interior Point OPTimizer for constrained optimization". In: *Optimization and Engineering* (2025), pp. 1–52. DOI: 10.1007/s11081-025-09975-y.

[17] EBatlleP. *Markov diagram v2*. Licensed under CC BY-SA 4.0; reproduced without modification. Wikimedia Commons. Jan. 23, 2020. URL: https://commons.wikimedia.org/wiki/File:Markov_diagram_v2.svg (visited on 08/20/2026).

[18] M. Towers et al. "Gymnasium: A standard interface for reinforcement learning environments". In: *arXiv preprint arXiv:2407.17032* (2024). DOI: 10.48550/arXiv.2407.17032. URL: https://arxiv.org/abs/2407.17032.

[19] A. Raffin et al. "Stable-Baselines3: Reliable reinforcement learning implementations". In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8. URL: https://www.jmlr.org/papers/v22/20-1364.html.